
Hardware/Software Design Description

For

CourtSweeper: A Vision-Based Tennis Ball Retriever with Dual-Roller Intake Mechanism

Version 1.0 approved

Group 13

Date 2026-03-23

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Revision History

Name	Date	Reason For Changes	Version
Yilin Zhang	2026-03-17	Initial document creation and LaTeX template configuration.	0.1
Yingrui Sun	2026-03-18	Drafted initial hardware specifications, including dual-roller mechanism and power system.	0.2
Shijia Luo	2026-03-18	Drafted software architecture, including machine vision algorithms and communication protocols.	0.3
Yuwei Zhao	2026-03-23	Comprehensive review and optimization. Finalized system design and approved Baseline 1.0.	1.0

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table of Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms and Abbreviations	2
1.4	References	3
1.5	Organization of Document	3
2	HARDWARE DESIGN	5
2.1	Camera Setup	5
2.2	DJI RoboMaster EP Chassis	6
2.3	Custom Intake Mechanism	6
2.4	LiDAR Sensor Setup	8
2.5	Jetson Orin NX Setup	8
2.6	Power Management System	9
3	SOFTWARE MODULES DESIGN	11
3.1	Architecture Overview	11
3.1.1	Chassis Navigation & DJI SDK Integration	11
3.1.2	SLAM & LiDAR Localization Package	12
3.1.3	Workspace Gridding & Calibration Package	12
3.1.4	Coverage Path Planning Package	12
3.1.5	Video I/O & Pre-processing Package	13
3.1.6	Video Processing Package (YOLO & OpenCV)	13
3.1.7	FSM-Based Event Recognition & Control Package	13
3.2	Class Design	14
4	DETAILED SOFTWARE DESIGN	15
4.1	Computer Vision & Object Detection Module	15
4.1.1	Module Architecture	15
4.1.2	Dataset Collection	15
4.1.3	Model Training	16
4.1.4	Cross-Platform Model Deployment	18
4.1.5	Real-Time Object Detection Pipeline	19
4.1.6	Visual Overlay & Debug Information	21
4.1.7	Inter-Module Data Exchange	22
4.1.8	Summary	22

4.2	Intake Motor Control Module	22
4.2.1	Hardware Driver Architecture	23
4.2.2	Low-Level Firmware Design	23
4.2.3	High-Level Control Interface	25
4.2.4	FSM Integration & Intake Trigger Logic	25
4.3	Proximity-Based Chassis Navigation Module	27
4.3.1	Distance Sensing Submodule	28
4.3.2	PID Controller Configuration	29
4.3.3	Chasing Sequence State Tracking	29
4.3.4	Multi-Stage Approach & Ingestion Logic	30
4.3.5	Chassis Drive Primitives	32
4.3.6	Idle & Search Behavior	32
5	USER INTERFACE DESIGN	34
5.1	Main Dashboard & Video HUD	34
5.2	Teleoperation & Control Panel	35
5.3	Telemetry & Log Console	35

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

1 INTRODUCTION

1.1 Purpose

This document describes the preliminary design of the CourtSweeper system. Its primary purpose is to provide a comprehensive architectural overview and detailed engineering specifications, serving as the Baseline 1.0 blueprint for the development team. It details the physical mechanical structures, electronic power systems, computer vision algorithms, and the iOS-based user interface to guide the implementation, integration, and testing phases.

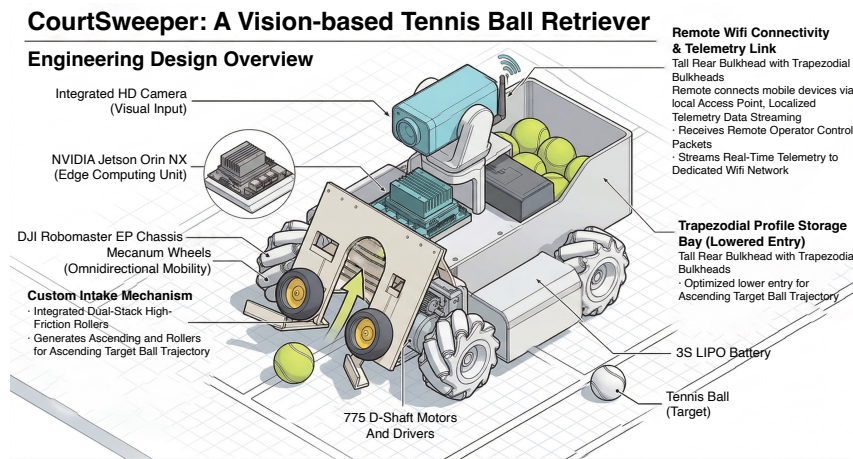


图 1: Comprehensive engineering design overview of the CourtSweeper robotic platform.

1.2 Scope

This document describes the design of the CourtSweeper, an intelligent, vision-guided robotic vehicle designed to automate the retrieval of scattered tennis balls in sports facilities. The scope of this system encompasses:

- **Hardware Subsystem:** A hybrid mobile platform utilizing the DJI RoboMaster EP [2] chassis to provide omnidirectional mobility and the primary HD camera feed. It is physically integrated with a custom-built, high-torque dual-roller intake mechanism, which is driven by 775 D-shaft motors and BTS7960 drivers for ball collection, and an NVIDIA Jetson Orin NX [4] acting as the central edge computing unit.
- **Software Subsystem:** An onboard AI pipeline utilizing the RoboMaster-SDK-Ultra [5] (a customized, high-performance fork of the official DJI SDK developed specifically for this project) for vehicle telemetry and camera stream extraction. This is combined with

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

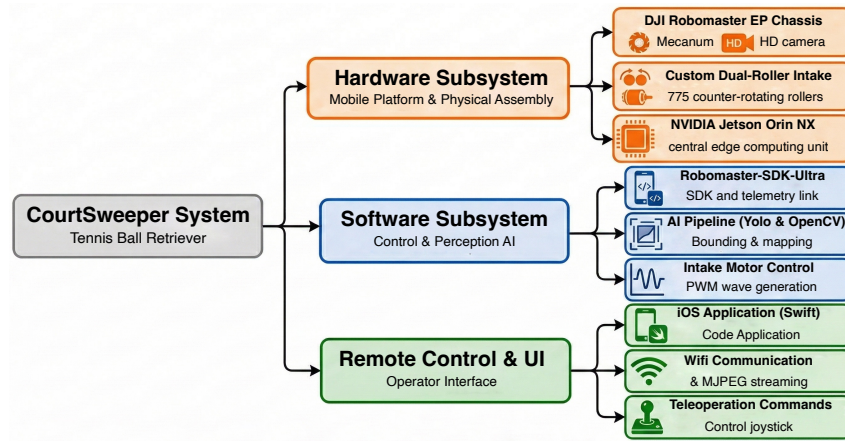


图 2: Hierarchical scope tree diagram detailing the overarching architecture of the CourtSweeper system.

the YOLO [6] object detection model and OpenCV, enabling real-time target recognition, spatial coordinate mapping, and dynamic intake motor control via PWM.

- **Remote Control & UI:** A native iOS application [1] developed in Swift, establishing a localized Wi-Fi connection to stream AI-annotated video feeds (via MJPEG) and transmit low-latency teleoperation commands to the Jetson unit.

1.3 Definitions, Acronyms and Abbreviations

表 1: List of abbreviations and terminology used in the CourtSweeper.

Term	Definition
HSDD	Hardware/Software Design Description.
CS	CourtSweeper, the designated project name for the tennis ball collecting robot.
UI	User Interface, specifically referring to the iOS application in this project.
YOLO	You Only Look Once (specifically yolo26), a SOTA real-time object detection system utilized for high-speed tennis ball recognition.
RoboMaster-SDK-Ultra	A customized, advanced fork of the official DJI SDK (developed by RamessesN), utilized to interface with the RoboMaster EP chassis, execute complex telemetry, and extract raw HD camera feeds.
OpenCV	Open Source Computer Vision Library, utilized for image processing, color space conversion, and bounding box coordinate extraction.
FSM	Finite State Machine, the logic architecture governing the autonomous and manual behavioral states of the robot.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table 1 continued

Term	Definition
IPC	Inter-Process Communication, a data exchange mechanism between OS processes. The custom SDK circumvents IPC to minimize video latency.
Mecanum Wheels	Omnidirectional wheels on the RoboMaster EP chassis that allow the robot to move in any direction without changing its orientation.
PWM	Pulse Width Modulation, utilized for custom motor speed regulation in the 775 motor-driven intake mechanism.
LiPo	Lithium Polymer, the high-discharge-rate battery chemistry utilized for the heavy-duty 3S actuation power source.
MJPEG	Motion JPEG, a video compression format used to stream AI-processed (bounding-box annotated) video frames to the iOS client with low latency.
HUD	Heads-Up Display, the primary transparent-style layout utilized in the iOS application for concurrent video and telemetry monitoring.
AP Mode	Access Point Mode, allowing the Jetson Orin NX to act as a localized Wi-Fi router for direct mobile connection in outdoor environments.

1.4 References

- [1] Apple Inc. *Apple Developer Documentation - Network Framework*. 2026. URL: <https://developer.apple.com/documentation>.
- [2] DJI. *RoboMaster EP Programming Guide and SDK Documentation*. 2020. URL: <https://www.dji.com/cn/robomaster-ep>.
- [3] *Dual Roller Mechanism*. 2024. URL: https://www.bilibili.com/video/BV1bgveeoEiX?vd_source=6621406a814999260edb3d6efe8942da.
- [4] Nvidia Inc. *NVIDIA Jetson Orin NX*. 2022. URL: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>.
- [5] RamessesN. *Robomaster SDK Ultra*. 2025. URL: <https://github.com/RamessesN/Robomaster-SDK-Ultra>.
- [6] *Ultralytics YOLO26*. 2026. URL: <https://docs.ultralytics.com/models/yolo26/>.

1.5 Organization of Document

To provide a structured and logical flow of the system architecture, this document is organized into the following major sections:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

表 2: Organization of this Hardware/Software Design Description.

Section	Core Content	Description
Section 2 <i>Hardware Design</i>	• DJI RoboMaster EP Chassis • Custom Intake Mechanism • Jetson Orin NX Setup • Power Management System	Provides the physical and electrical blueprint of the robot. It details the integration of the hybrid mobile platform, the custom 775 motor-driven dual-roller mechanism, and the multi-voltage power distribution topology.
Section 3 <i>Software Design</i>	• DJI SDK Integration • Video Pre-processing • YOLO and OpenCV Pipeline • FSM-Based Control Logic • Class Design & Architecture	Describes the overarching onboard AI architecture. It breaks down the integration of the custom SDK for chassis navigation, the computer vision pipeline for target detection, and the finite state machine governing the intake motors.
Section 5 <i>UI Design</i>	• Main Dashboard & Video HUD • Teleoperation & Control Panel • Telemetry & Log Console	Outlines the Swift-based iOS application architecture, detailing the visual presentation layer, network socket communication, and real-time system monitoring protocols.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

2 HARDWARE DESIGN

The hardware architecture of the CourtSweeper system is a hybrid design, integrating a commercial robotic platform with custom-engineered mechanical and computational subsystems. This section details the physical embodiment, sensor configurations, and electrical topologies required to achieve autonomous tennis ball retrieval.

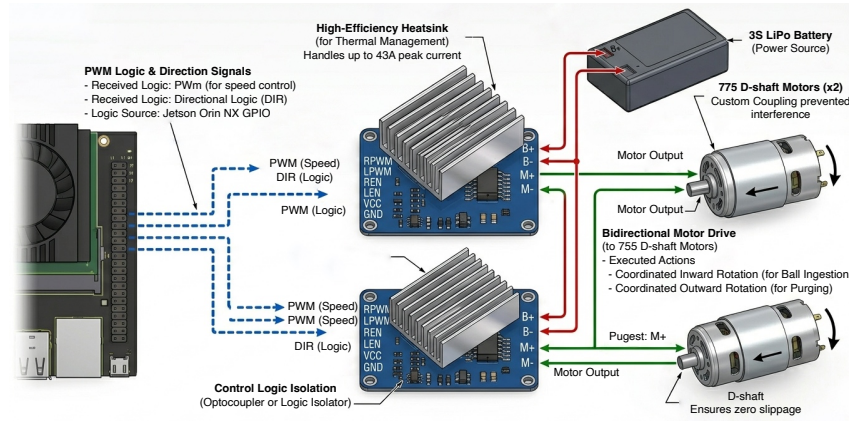


图 3: An engineering diagram detailing the BTS7960 high-current drive architecture for the CourtSweeper system, illustrating the connection of an NVIDIA Jetson Orin NX to two BTS7960 motor drivers powered by a 3S LiPo battery to control two 775 D-shaft motors, featuring logic signal connections, power distribution, bidirectional motor drive, and technical callouts for key components like heatsinks and motor features.

2.1 Camera Setup

The primary visual input via the integrated HD camera on the Robomaster EP chassis, which feeds raw video streams to the edge computing unit for target detection. The core hardware parameters of the integrated camera are detailed in Table 3.

表 3: DJI RoboMaster EP Integrated Camera Specifications

Hardware Parameter	Specification
Sensor Type	CMOS 1/4", Effective Pixels: 5 MP
Field of View (FOV)	120°
Max Video Resolution	FHD: 1080p@30fps / HD: 720p@30fps
Max Video Bitrate	16 Mbps
Max Still Photo Resolution	2560 × 1440

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026



图 4: A physical picture of Robomaster EP Camera Module.

2.2 DJI RoboMaster EP Chassis



图 5: A physical picture of Robomaster EP Chassis Module.

The foundation of the CourtSweeper is the DJI RoboMaster EP chassis. Selected for its industrial-grade stability and payload capacity, the chassis is equipped with four Mecanum wheels, granting the robot true omnidirectional mobility. This holonomic drive system is crucial for the precise spatial adjustments required to align the front-facing intake mechanism with scattered tennis balls without needing a turning radius.

2.3 Custom Intake Mechanism

To achieve reliable and rapid ball collection, a custom dual-roller intake mechanism [3] was designed and mounted at the front of the chassis. The actuation is driven by two high-torque

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

775 DC motors (D-shaft configuration, rated at 12V and 4600 RPM).

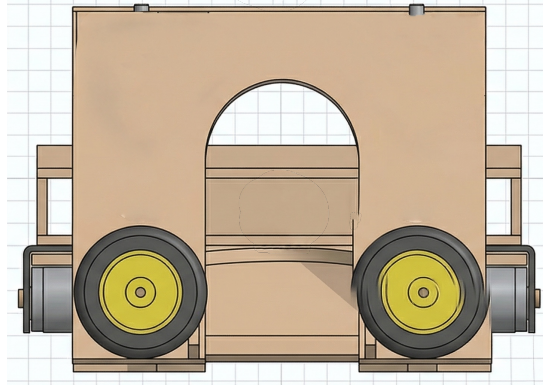


图 6: A detailed front-on view of the Courtsweeper dual-roller intake mechanism.

- **Mechanical Coupling:** The motors are mated to 60mm solid rubber wheels featuring high-friction hexagonal grooves via 30mm extended brass couplings. This extended design provides essential clearance to prevent mechanical interference between the rotating wheel hubs and the motor casing, while the D-shaft ensures zero slippage under high torque.
- **Motor Drivers:** Two BTS7960 high-current motor driver modules (capable of handling up to 43A peak current) are utilized to independently control the two motors. They receive Pulse Width Modulation (PWM) and directional logic signals from the central computing unit to execute coordinated inward rotation (for ball ingestion) or outward rotation (for purging).

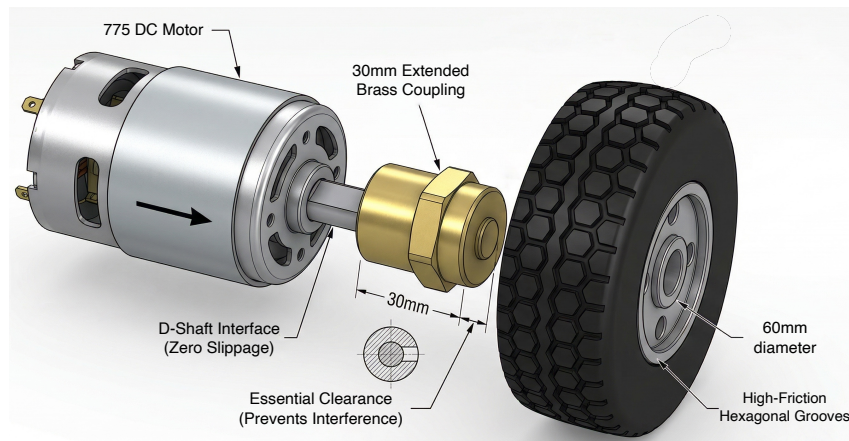


图 7: Close-up view of the mechanical coupling. The 30mm extended brass coupling provides critical operational clearance between the 775 motor casing and the high-friction intake roller.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

2.4 LiDAR Sensor Setup

To ensure precise operational boundary gridding and efficient path planning (as outlined in the software architecture Fig.11), the Courtsweeper incorporates a dedicated 2D LiDAR sensor. Based on a triangulation principle, this sensor provides robust 2D scanning of the court environment and potential obstacles.

- **Sensor Details:** The selected hardware is the YDLIDAR X3-YB-1 triangulation-based LiDAR. It is mounted at the rear of the chassis and interfaces with the Jetson Orin NX via a customized USB-serial connection.
- **Specifications:** The X3-YB-1 offers a maximum distance range of up to 10 meters, enabling comprehensive court coverage. Key operational parameters include a sampling frequency of 4000 Hz, a standard rotating frequency of 7 Hz, and an angular resolution of 0.6° .



图 8: A physical picture of the YDLIDAR X3-YB-1 LiDAR sensor.

2.5 Jetson Orin NX Setup

The cognitive core of the system is the NVIDIA Jetson Orin NX. Acting as the Edge Computing Unit, it is responsible for handling computationally intensive tasks locally, thereby eliminating cloud-dependency and minimizing latency. The Jetson module runs the Ubuntu operating system and is configured to:

1. Execute the YOLO object detection pipeline concurrently with OpenCV operations to process the H.264 video stream captured by the DJI chassis.
2. Run the customized RoboMaster-SDK-Ultra to translate bounding box coordinates into proportional navigation commands.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

3. Generate precise PWM signals via its GPIO pins to command the BTS7960 drivers.
4. Operate in Access Point (AP) mode to establish a localized Wi-Fi network for the iOS UI client.



图 9: A physical picture of NVIDIA jetson Orin NX.

2.6 Power Management System

Given the diverse voltage requirements and the risk of electromagnetic interference from high-current motors, the CourtSweeper employs an isolated, multi-rail power distribution topology:

- **Mobility Power:** The DJI RoboMaster EP chassis and its onboard camera are powered autonomously by the proprietary DJI Intelligent Battery (3S LiPo, Nominal 10.8V, 2400mAh, 25.92Wh).
- **Actuation Power:** The 775 intake motors and BTS7960 drivers are powered by a dedicated 3S (11.1V) Lithium Polymer (LiPo) battery rated at 5200mAh and 40C discharge. This ensures the motors receive sufficient burst current during ball compression without inducing voltage sags in the logic circuits. A low-voltage buzzer (BX100) is attached to the balance lead for real-time cell monitoring.
- **Logic Power:** The Jetson Orin NX requires a stable 19V input. To prevent logic resets caused by motor back-EMF, it is powered by a galvanically isolated power source.
- **Common Grounding:** Crucially, the ground (GND) planes of the Jetson Orin NX and the BTS7960 motor drivers are tied together to ensure reliable logic-level signal transmission.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

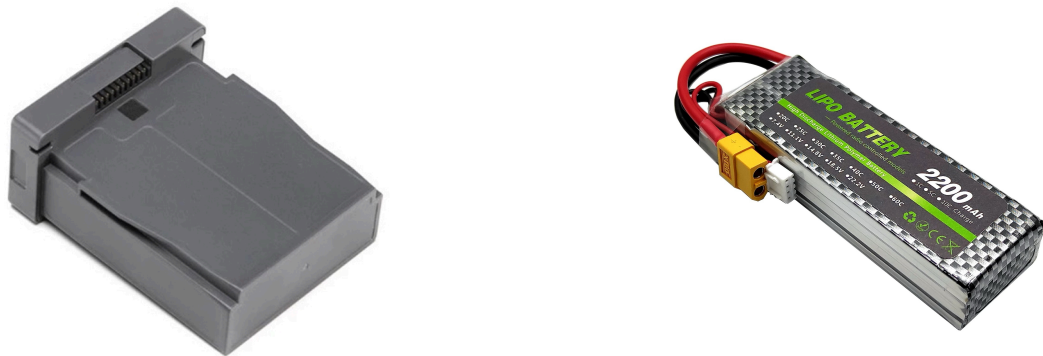


图 10: A physical picture of Robomaster EP battery & Motor-driven 3S LiPo battery.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

3 SOFTWARE MODULES DESIGN

3.1 Architecture Overview

The software subsystem of CourtSweeper operates primarily on the NVIDIA Jetson Orin NX edge computing unit, utilizing a multi-threaded pipeline designed for high-throughput and low-latency operation. The architecture integrates heterogenous sensory inputs—comprising the high-definition DJI camera stream and 2D LiDAR range data—to establish full situational awareness.

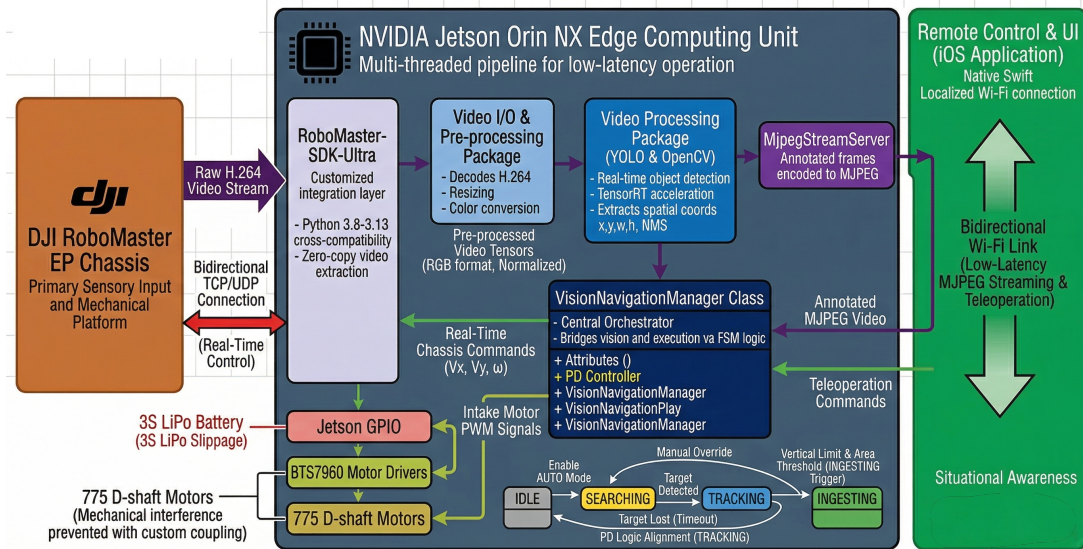


图 11: Detailed comprehensive diagram of the CourtSweeper software subsystem architecture, integrating visual perception and SLAM-based navigation pipelines.

The core logic bridges these sensory inputs with the mechanical outputs (Mecanum chassis navigation and 775 motor actuation) through an advanced decision-making engine. This engine combines real-time SLAM (Simultaneous Localization and Mapping), heuristic workspace grid-ding, complete coverage path planning, and deep learning-based object detection, all governed by a robust Finite State Machine (FSM).

3.1.1 Chassis Navigation & DJI SDK Integration

To interface with the mobile platform, the system implements the **RoboMaster-SDK-Ultra**, a customized integration layer offering native cross-compatibility across Python 3.8 ~ 3.13. This allows DJI hardware drivers to coexist in the same execution environment as modern

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

YOLO, OpenCV, and ROS-based SLAM pipelines, circumvents severe Inter-Process Communication (IPC) overhead when transferring large data matrices.

Operationally, this unified SDK manages a high-speed connection over the local network. It directly extracts the raw H.264 video stream from the integrated camera for zero-copy visual processing, while concurrently transmitting translational (V_x, V_y) and rotational (ω) velocity commands back to the holonomic drive system based on calculated global paths or local reactive maneuvers.

3.1.2 SLAM & LiDAR Localization Package

To provide the robot with precise spatial awareness, this package interfaces with the LiDAR via USB-serial. It implements a robust SLAM algorithm accelerated by the Jetson's GPU to process scan matching in real-time. This module is responsible for:

Map Generation:

Constructing a 2D occupancy grid map of the surrounding environment during the initial setup phase.

Real-time Localization:

Providing continuous, high-frequency estimates of the robot's pose (x, y, θ) within the generated map by fusing LiDAR odometry and chassis wheel odometry data.

3.1.3 Workspace Gridding & Calibration Package

This module is utilized during system initialization to define the operational boundaries.

- **Court Calibration:** The operator manually drives the robot along the perimeter of the tennis court playing area. The package records these pose boundaries within the SLAM map to isolate the sweeping workspace.
- **Grid Discretization:** The calibrated workspace is discretized into a uniform grid map. Each grid cell dimensions correspond to the sweeping width of the intake mechanism. Cells are classified as occupied (by static obstacles), traversable, cleared, or containing a target (projected from visual data).

3.1.4 Coverage Path Planning Package

This package is the core efficiency optimizer. In the absence of visually detected tennis balls, it ensures the entire traversable court area is swept systematically.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

- **Global Planning:** Utilizing algorithms such as A* or Spanning Tree Coverage, it generates an optimal coverage path through all traversable grid cells to minimize sweeping time and redundant traversal.
- **Local Planning & Replenishment:** If visual processing detects a ball, the global coverage path is paused. After ball ingestion, the planner calculates a local trajectory back to the last paused waypoint on the global coverage path to resume systematic sweeping.

3.1.5 Video I/O & Pre-processing Package

Before AI inference, the raw H.264 video feed is decoded into discrete frame matrices. The pre-processing pipeline scales the HD frames to specified YOLO input dimensions and converts color spaces to RGB, normalizing pixel values to a $[0, 1]$ range for optimized neural network ingestion.

3.1.6 Video Processing Package (YOLO & OpenCV)

Utilizing the YOLO architecture accelerated by TensorRT, this module performs real-time object detection to identify tennis balls. OpenCV functions extract the spatial coordinates of the bounding box ($x, y, width, height$) and calculate the target centroid.

Crucially, by utilizing the current robot pose from the SLAM package and known camera intrinsic parameters, this module projects the image-plane coordinates onto the ground plane to estimate the ball's (x, y) location in the global map, updating the target status of specific grid cells.

3.1.7 FSM-Based Event Recognition & Control Package

The robot's behavioral logic is strictly governed by a Finite State Machine (FSM), decoupling perception from mechanical actuation. The optimized state transitions are as follows:

- **State 0 - IDLE:** Awaiting commands. Manual teleoperation via iOS UI overrides all behaviors. Court calibration is performed in this state.
- **State 1 - SEARCHING (Optimized Coverage):** If no ball is visually detected, the FSM enters searching state. Instead of random patrol, the chassis follows the pre-generated **Complete Coverage Path**. The system continuously ticks off grid cells as "cleared" upon traversal.
- **State 2 - TRACKING (Target Alignment):** Upon visual detection, FSM transitions to Tracking. Global path following is paused. A PD controller uses the camera centroid offset for rapid alignment, while a local planner drives the chassis toward the projected map coordinates of the ball.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

- **State 3 - INGESTING (Intake Trigger):** As defined previously, triggered by bounding box area thresholds indicating physical contact. Motors activate via PWM. Upon successful ingestion, the corresponding grid cell is marked as "cleared," and the FSM transitions back to SEARCHING, utilizing the CPP package to return to the paused coverage path.

3.2 Class Design

CourtNavigationManager Class Description

To support the integrated SLAM and Planning pipelines, the central orchestrator is upgraded to the CourtNavigationManager. It bridges visual perception, spatial mapping, grid-ding, and path planning modules.

Attribute	Description
Class Name	CourtNavigationManager
Aggregated Classes	YoloDetector, SlamLocalizer, GridMapHandler, CoveragePathPlanner, DjiChassisController, MotorPwmDriver
Associated Classes	MjpegStreamServer, RosBridgeServer (for LiDAR data)
Data Members	• <code>current_state</code>: Enum (IDLE, CALIBRATING, SEARCHING, TRACKING, INGESTING). • <code>robot_pose_global</code>: Pose (x, y, θ) provided by SlamLocalizer. • <code>discretized_grid</code>: 2D Array/Structure managing cell states (Occupied, Swept, Ball). • <code>global_coverage_path</code>: List of Waypoints to ensure 100% coverage.
Member Functions	• <code>update_master_state()</code>: High-level FSM logic switching based on vision, path progress, and grid status. • <code>calibrate_court_perimeter()</code>: Records boundary poses during manual driving to define grid limits. • <code>generate_gridded_workspace()</code>: Discretizes calibrated area and calls CoveragePathPlanner for the initial path. • <code>compute_navigation_command()</code>: In SEARCHING, computes commands to follow the coverage path. In TRACKING, switches to PD-based visual servoing and projected coordinate driving.
Processing	Operates in a dedicated high-priority thread. It fuses pose data and vision data to update the grid map, executes the FSM, dispatches velocity commands to the DJI SDK, and triggers intake motors.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4 DETAILED SOFTWARE DESIGN

4.1 Computer Vision & Object Detection Module

This module is the core perception layer of the CourtSweeper system, responsible for real-time video frame capture, tennis ball target detection, and providing pixel-space coordinate offset to the downstream motion control module. The system is built on the YOLO (You Only Look Once) deep learning object detection framework, supports multi-platform deployment (macOS, Linux/Jetson), and is optimized with TensorRT acceleration for embedded edge computing scenarios.

4.1.1 Module Architecture

The overall workflow of the computer vision module can be summarized into four stages: dataset collection and annotation, model training, cross-platform model deployment, and real-time object detection. The system receives the real-time video stream from the RoboMaster gimbal camera, performs YOLO inference to output the bounding box coordinates of tennis ball targets, and then computes the horizontal offset of the target relative to the image center (visual error), which serves as the input signal for the chassis and gimbal control loops.

The module consists of the following core files:

<code>cv_config.py</code>	Device-adaptive and path configuration
<code>dataset_collect.py</code>	Dataset collection tool
<code>model_training.py</code>	YOLO model training script
<code>pt2onnx.py</code>	PyTorch model export to ONNX format
<code>onnx2engine.sh</code>	ONNX model conversion to TensorRT engine
<code>video_node.py</code>	Real-time object detection main node

4.1.2 Dataset Collection

The quality of the dataset directly affects the detection accuracy of the model. We developed the `dataset_collect.py` tool to capture tennis ball images in actual court environments using the built-in RoboMaster camera.

The collection program communicates with the robot via a direct Wi-Fi connection (AP mode), starts a 720P HD video stream, and displays the resized frames at 1280×960 pixels in real time. A crosshair is drawn at the center of the frame to assist composition, helping the collector position the tennis ball in the central region. Pressing the `s` key triggers a snapshot save, followed by a brief white flash as visual feedback; pressing `q` exits the program. Images

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

are saved to the `./dataset/tennis_v2/` directory with a timestamp-plus-sequence naming scheme (e.g., `tennis_20250519_143025_0001.jpg`).

```

1 def img_collect():
2     save_dir = "./dataset/tennis_v2"
3     os.makedirs(save_dir, exist_ok=True)
4
5     ep_robot = robot.Robot()
6     ep_robot.initialize(conn_type="ap")
7     ep_robot.set_robot_mode(mode="free")
8     ep_camera = ep_robot.camera
9     ep_camera.start_video_stream(display=False,
10                                 resolution=camera.STREAM_720P)
11
12     count = 0
13     while True:
14         img = ep_camera.read_cv2_image()
15         if img is not None:
16             img_resized = cv2.resize(img, (1280, 960))
17             cv2.line(display_img, (center_w - 20, center_h),
18                     (center_w + 20, center_h), (0, 255, 0), 2)
19             cv2.line(display_img, (center_w, center_h - 20),
20                     (center_w, center_h + 20), (0, 255, 0), 2)
21             if key == ord('s'):
22                 cv2.imwrite(filename, img_resized)
23

```

After collection, annotation tools such as Labellmg are used to label the tennis balls in the images with bounding boxes, generating YOLO-format label files (`.txt`). A `data.yaml` dataset configuration file is then written, specifying the training and validation set paths and class names.

4.1.3 Model Training

The object detection model is trained using the Ultralytics YOLO framework. Starting from the YOLOv6-Nano (`yolo26n.pt`) pre-trained weights as a foundation, transfer learning is applied to rapidly adapt the model from general-purpose detection capability to the single tennis ball class.

The training hyperparameters are configured as follows:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Initial learning rate (lr0) 0.01
final learning rate (lrf) 0.01
Optimizer auto (SGD with momentum=0.937)
Weight decay 0.0005
Warmup epochs 3.0
Data augmentation Mosaic stitching, random horizontal flip (fliplr=0.5), HSV color perturbation, random affine transformation (scale=0.5, translate=0.1), RandAugment automatic augmentation, random erasing (erasing=0.4)
Early stopping patience 100 epochs

```

1 from ultralytics import YOLO
2
3 def train_model():
4     model = YOLO("./weight/yolo26n.pt") # load pre-trained model
5
6     results = model.train(
7         data=DATA_YAML, # dataset configuration file
8         epochs=200, # training epochs
9         imgsz=640, # input image size
10        batch=16, # batch size (adjust by GPU memory)
11        name=MODEL_NAME, # model name
12        device=DEVICE, # runtime device
13        workers=2, # data loading threads
14        project="./runs/train_tennis", # output directory
15        exist_ok=False # do not overwrite existing experiment
16    )

```

Device selection is handled by the adaptive logic in `cv_config.py`, which probes hardware accelerators in priority order: CUDA (NVIDIA GPU), MPS (Apple Silicon), XPU (Intel GPU), and falls back to CPU if none is available:

```

1 DEVICE: str = (
2     "cuda" if torch.cuda.is_available() else
3     "mps" if torch.mps.is_available() else
4     "xpu" if torch.xpu.is_available() else
5     "cpu"
6 )

```

The artifacts produced during training include: the best-weight file `best.pt`, the last-epoch weight `last.pt`, F1 curve plot, PR curve plot, confusion matrix, training batch visualization

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

samples, and a `results.csv` file recording per-epoch losses and metrics for subsequent analysis and tuning.

4.1.4 Cross-Platform Model Deployment

To accommodate the inference performance requirements of different deployment environments, the system implements a model conversion pipeline from PyTorch to ONNX to TensorRT.

PyTorch → ONNX Conversion: Using the built-in export functionality of Ultralytics, the `.pt` weights can be converted to the open ONNX intermediate representation format with a single line of code:

```
1 from ultralytics import YOLO
2
3 model = YOLO("./pt/tennis.pt")
4 model.export(format="onnx")
```

The generated `.onnx` file can be used for inference on any platform supporting ONNX Runtime, ensuring the model's universality and portability.

ONNX → TensorRT Engine: On edge computing platforms such as the NVIDIA Jetson, to fully leverage GPU inference acceleration, NVIDIA TensorRT's `trtexec` tool is used to compile the ONNX model into a highly optimized TensorRT engine (`.engine` file), with FP16 half-precision inference enabled to substantially boost the inference frame rate with negligible loss in detection accuracy:

```
1 /usr/src/tensorrt/bin/trtexec \
2 --onnx=onnx/tennis_v2.onnx \
3 --saveEngine=engine/tennis_v2.engine \
4 --fp16
```

At inference time, the appropriate model format is automatically selected based on the runtime platform:

```
1 # Mac Config
2 model = YOLO("src/vision/mlmodel/pt/tennis_v2.pt")
3
4 # Linux Config (Jetson + TensorRT)
5 # model = YOLO("src/vision/mlmodel/engine/tennis_v2.engine")
6 # model.names = {0: 'tennis ball'}
```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.1.5 Real-Time Object Detection Pipeline

`video_node.py` is the runtime core of the entire vision module. It is responsible for acquiring video frames from the robot camera, invoking the YOLO model for object detection, computing the target's offset from the frame center, and overlaying rich debugging information on the frame.

Video Stream Acquisition.

The video stream is started at 360P resolution via the RoboMaster SDK to balance image quality and frame rate. Each received frame is resized to 640×360 pixels before being fed into the YOLO model for processing.

```

1  def video_capture(ep_camera):
2      global latest_frame, annotated_frame, running
3
4      ep_camera.start_video_stream(display=False,
5                                   resolution=camera.STREAM_360P)
6
7      while running:
8          img = ep_camera.read_cv2_image()
9          if img is not None:
10             img = cv2.resize(img, (640, 360))
11             latest_frame = img
12             annotated_frame = yolo_predict(img)
13             cv2.imshow("on Live", annotated_frame)

```

Object Detection and Filtering.

YOLO inference uses a confidence threshold of 0.25 to filter out low-quality detections. When multiple detection results are present in the frame (e.g., multiple tennis balls), the system compares the bottom Y-coordinate (y_2 value) of each bounding box to select the ball closest to the camera—a larger y_2 value indicates that the target is lower in the frame, i.e., nearer to the camera. This strategy ensures that the robot prioritizes locking onto and tracking the nearest target ball.

Once the target is selected, the horizontal coordinate of its bounding box center, `target_x`, is subtracted from the frame centerline `center_x` to compute the horizontal visual deviation `visual_error_x`. This offset is the key input quantity for subsequent chassis rotation control:

$$\text{visual_error_x} = \text{target_x} - \frac{\text{width}}{2}$$

```

1  def yolo_predict(frame):

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

2     results = model(frame, conf=0.25, device=cfg.DEVICE,
3                       verbose=False)
4     annotated = results[0].plot()
5
6     height, width = frame.shape[:2]
7     center_x = width // 2
8
9     if len(results[0].boxes) > 0:
10         boxes = results[0].boxes
11
12         closest_box = None
13         max_y2 = -1
14         for box in boxes:
15             x1, y1, x2, y2 = map(int, box.xyxy[0])
16             if y2 > max_y2:
17                 max_y2 = y2
18                 closest_box = box # select the nearest target
19
20         x1, y1, x2, y2 = map(int, closest_box.xyxy[0])
21         target_x = (x1 + x2) // 2
22         target_y = (y1 + y2) // 2
23         visual_error_x = target_x - center_x

```

Target Locking and Scanning Mechanism.

The system maintains three states: Locked (target locked), Searching (normal search in progress), and Scanning (360-degree scan for ball search). When no tennis ball is detected for 15 consecutive frames (`no_ball_counter >= 15`), the system automatically triggers the `scan_needed` flag, notifying the chassis control module to perform an in-place rotation scan to re-locate the ball. Upon detecting a target, the counter resets to zero and the state returns to locked. Additionally, if the target approaches the edge of the frame (within 10 pixels of the boundary), the system treats it as about to leave the field of view and actively relinquishes the lock to avoid ineffective tracking.

```

1     if not (10 <= target_x <= width - 10) or \
2           not (10 <= target_y <= height - 10):
3         target_x = None
4         target_y = None
5         visual_error_x = 0
6         target_locked = False
7
8     # no ball detected
9     no_ball_counter += 1

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

10 if no_ball_counter >= 15: # 15 consecutive frames without target → trigger scan
    ↪ mode
11     scan_needed = True

```

4.1.6 Visual Overlay & Debug Information

To facilitate real-time debugging and status monitoring, the system overlays the following information on each frame in OSD (On-Screen Display) format:

- **Resolution (RES):** Current frame dimensions, displayed in red
- **Frame Rate (FPS):** Sliding-average frame rate over the last 10 frames, displayed in green
- **Distance:** Real-time ranging value from the distance sensor module (unit: cm), displayed in blue
- **Status:** Current tracking state —Locked (green) / Searching or Scanning (orange)
- **Center Crosshair:** A gray crosshair marking the geometric center of the frame
- **Target Marker:** When a target is locked, a solid blue circle is drawn at the target center with the horizontal offset value annotated beside it
- **Bounding Box:** YOLO detection boxes are rendered as yellow rectangles

```

1 # Resolution
2 cv2.putText(annotated, f"RES: {width}x{height}",
3             (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
4             1, (0, 0, 255), 2, lineType=cv2.LINE_AA)
5 # Frame Rate
6 cv2.putText(annotated, f"FPS: {int(avg_fps)}",
7             (10, 60), cv2.FONT_HERSHEY_SIMPLEX,
8             1, (0, 255, 0), 2, lineType=cv2.LINE_AA)
9 # Distance
10 cv2.putText(annotated, f"Distance: {distance_text}",
11            (10, 90), cv2.FONT_HERSHEY_SIMPLEX,
12            1, (255, 0, 0), 2, lineType=cv2.LINE_AA)
13 # Status
14 state_text = "Locked" if target_locked else \
15             ("Scanning" if scan_needed else "Searching")
16 cv2.putText(annotated, f"Status: {state_text}",
17            (10, 120), cv2.FONT_HERSHEY_SIMPLEX,
18            1, color, 2, lineType=cv2.LINE_AA)

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.1.7 Inter-Module Data Exchange

The computer vision module shares data with other system modules through module-level global variables:

- `visual_error_x`: The horizontal pixel offset of the target relative to the frame center, used by the chassis control module (`chassis_ctrl.py`) to compute the rotational angular velocity
- `target_locked`: The target lock status flag, used by the main control logic to determine the current tracking phase
- `scan_needed`: The 360-degree scan ball-search request flag, triggering the chassis to execute a scanning rotation maneuver
- `latest_frame` / `annotated_frame`: The raw and annotated frames, available for other modules that require image data

Simultaneously, the vision module also reads the `latest_distance` variable provided by the distance sensor module, displaying real-time tennis ball distance information on the OSD and thereby achieving fused multi-sensor information presentation.

4.1.8 Summary

The CourtSweeper computer vision module utilizes the YOLO deep learning framework to achieve a complete closed-loop pipeline encompassing data collection, model training, cross-platform deployment, and real-time inference. Through the nearest-target filtering strategy, target locking and scanning search mechanism, and horizontal visual deviation computation, the module provides reliable perceptual input for the robot's autonomous tracking and ball retrieval behaviors. The model deployment pipeline supports the three-stage PyTorch→ONNX→TensorRT conversion, enabling efficient inference at FP16 half-precision on edge devices such as the NVIDIA Jetson, thereby satisfying real-time performance requirements.

4.2 Intake Motor Control Module

The intake motor control module is the critical actuation unit of the CourtSweeper system. It is responsible for driving the dual-roller mechanical structure to ingest the ball into the onboard storage bay once the vision system has locked onto a tennis ball target. This module spans both low-level hardware driving and high-level decision logic, forming a complete control chain from signal generation to intelligent triggering.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.2.1 Hardware Driver Architecture

The intake mechanism is driven by two 775 DC motors (D-shaft configuration, rated at 12 V and 4600 RPM), each coupled to a 60 mm high-friction solid rubber roller via a 30 mm extended brass coupling. Motor driving is handled by two BTS7960 high-current full-bridge driver modules (capable of handling up to 43 A peak current), independently controlling the direction and speed of the left and right rollers.

The BTS7960 driver modules interface with the lower-level controller via a three-wire interface (enable pin EN, forward PWM pin RPWM, and reverse PWM pin LPWM). The system utilizes a dedicated 3S (11.1 V) 5200 mAh Lithium Polymer (LiPo) battery (discharge rate 40C) to power the driver modules, ensuring sufficient burst current for the motors during ball compression while preventing voltage sag interference to the main control logic circuits. Low-level PWM signal generation is handled by an Arduino microcontroller, which receives commands from the Jetson Orin NX via USB serial.

4.2.2 Low-Level Firmware Design

The low-level Arduino firmware (`roller_setting.ino`) is responsible for converting serial commands into PWM signals, enabling real-time control of the dual BTS7960 drivers. The firmware defines the control interfaces for the left and right motors through a pin-mapping scheme, as detailed in Table 4.

表 4: 775 Motor Driver Pin Mapping Table

Motor	Enable (EN)	Forward PWM (RPWM)	Reverse PWM (LPWM)
Left Motor	Pin 4	Pin 5	Pin 6
Right Motor	Pin 7	Pin 9	Pin 10

The firmware main loop operates in blocking serial-read mode: when data is available in the serial buffer, it calls `Serial.parseInt()` to continuously parse the left and right motor speed values (formatted as "L_speed,R_speed\n"), then invokes the `driveMotor()` function to convert the values into corresponding PWM signals. This function clamps the speed to the range $[-255, 255]$ and determines the rotation direction based on the sign: positive values activate the RPWM channel, negative values activate the LPWM channel, and a zero value pulls the enable pin low to cut off the output.

```

1 void driveMotor(int enPin, int fwdPin, int revPin, int speed) {
2     speed = constrain(speed, -255, 255);
3
4     if (speed == 0) {

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

5     digitalWrite(enPin, LOW);
6     analogWrite(fwdPin, 0);
7     analogWrite(revPin, 0);
8 } else if (speed > 0) {
9     digitalWrite(enPin, HIGH);
10    analogWrite(fwdPin, speed);
11    analogWrite(revPin, 0);
12 } else {
13     digitalWrite(enPin, HIGH);
14     analogWrite(fwdPin, 0);
15     analogWrite(revPin, abs(speed));
16 }
17 }

```

To ensure system safety, the firmware incorporates a communication timeout protection mechanism. Upon each successful command parsing, a timestamp is recorded. If no new command is received within 500 ms while the motors are in motion, `stopMotor()` is automatically invoked to zero all motor outputs, preventing the motors from running idle during a communication loss.

```

1 void loop() {
2     if (Serial.available() > 0) {
3         int speedL = Serial.parseInt();
4         int speedR = Serial.parseInt();
5
6         if (Serial.read() == '\n') {
7             driveMotor(L_EN, L_RPWM, L_LPWM, -speedL);
8             driveMotor(R_EN, R_RPWM, R_LPWM, speedR);
9
10            lastRecvTime = millis();
11
12            if (speedL != 0 || speedR != 0) {
13                isMoving = true;
14            }
15        }
16    }
17
18    if (isMoving && (millis() - lastRecvTime > 500)) {
19        stopMotor();
20        isMoving = false;
21    }
22 }

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.2.3 High-Level Control Interface

On the Jetson Orin NX side, the Python module `roller_drive.py` encapsulates serial communication with the Arduino. Its core is the `RollerMotor` class: the constructor establishes a connection via the `PySerial` library at 115200 baud and waits 2 s for initialization, while the `set_speed()` method formats the left and right wheel speeds as a CSV string and writes it to the serial port.

```

1 class RollerMotor:
2     def __init__(self, port='', baudrate=115200):
3         logger.info(f"Connecting to motor on {port} ...")
4         try:
5             self.ser = serial.Serial(port, baudrate, timeout=1)
6             time.sleep(2)
7             logger.info("Connection successful!")
8         except Exception as e:
9             logger.error(f"Failed to connect to serial port. Error details: {e}")
10            exit(1)
11
12    def set_speed(self, left_speed, right_speed):
13        command = f"{int(left_speed)},{int(right_speed)}\n"
14        self.ser.write(command.encode('utf-8'))
15        logger.info(f"Roller Speed: L = {left_speed}, R = {right_speed}")
16
17    def close(self):
18        self.set_speed(0, 0)
19        time.sleep(0.1)
20        self.ser.close()
21        logger.info("Connection closed.")

```

This design treats the Python side as a pure command-dispatching layer, offloading all PWM timing generation and direction logic to the Arduino firmware, allowing the upper layer to focus on decision scheduling without concern for time-sensitive low-level signal generation. The `close()` method ensures safe shutdown by issuing a zero-speed command before closing the serial port.

4.2.4 FSM Integration & Intake Trigger Logic

The motor control module is integrated into the chassis control main loop (`chassis_ctrl.py`) and is triggered by the Finite State Machine (FSM) based on visual perception results. The triggering and execution of the intake logic can be decomposed into the following three phases:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Phase 1: Target Distance Pre-Assessment

When the vision system locks onto a tennis ball target (`target_locked = True`), the system enters the chasing sequence (`is_chasing_sequence`). The PD controller drives the chassis to continuously align with and approach the target, while simultaneously reading the front-facing distance sensor value in real time. When the target distance falls within 200 mm, the system determines that the ball has entered the reachable range of the rollers and immediately transitions to the pre-spin phase.

Phase 2: Motor Pre-Spin & Slow Advance

Upon entering the pre-spin phase, the motors begin rotating at a medium speed (both set to 90, approximately 35% duty cycle), while the chassis continues a slow forward advance at speed 30. This design ensures that the rollers already possess rotational kinetic energy at the moment of contact with the ball, preventing jamming or bouncing caused by acceleration from a standstill.

```

1  if dist <= 200:
2      logger.info(f" 目标进入预备区 ({dist/10:.1f}cm)! 预启动电机并缓慢推进...")
3
4      if intake_motor:
5          intake_motor.set_speed(90, 90)
6
7      drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)

```

Phase 3: Ingestion Confirmation & Timeout Protection

After the motors start, the system enters an ingestion confirmation window of at most 3 s. The front-facing distance sensor value is continuously monitored at a 20 ms sampling period: if the ball is successfully ingested, the distance sensor reading will jump beyond the measurement range (> 250 mm). The system performs 10 consecutive disappearance confirmations (totaling 200 ms) to prevent false positives. Once the ball is confirmed ingested, it immediately stops the chassis and motors and resets the chasing state, returning the FSM to the searching state to resume the coverage path.

If the distance sensor reading does not exhibit any jump within the 3 s window, the system determines that a jam or missed-ball anomaly has occurred and likewise stops execution while outputting a warning log, preventing motor overheating from prolonged stalling.

```

1  swallow_start_time = time.time()
2  missing_count = 0
3
4  while time.time() - swallow_start_time < 3.0:

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

5     current_dist = ds.latest_distance if ds.latest_distance is not None else 8848
6
7     if current_dist > 250:
8         missing_count += 1
9     else:
10        missing_count = 0
11
12    if missing_count >= 10:
13        is_swallowed = True
14        break
15
16    time.sleep(0.02)
17
18    if is_swallowed:
19        logger.info(" 球已成功吞入网箱！ ")
20    else:
21        logger.info(" 推进超时，发生卡球或漏球。")
22
23    chassis_stop(ep_chassis)
24
25    if intake_motor:
26        intake_motor.set_speed(0, 0)

```

Blind-Zone Relay Mechanism

Additionally, the system implements a blind-zone relay logic: when the ball distance has already been reduced to within 300 mm but the visual target is briefly lost (with the distance sensor still reading within the valid 450 mm range), the chassis maintains straight forward advance without immediately aborting the chasing sequence. This mechanism effectively compensates for the visual blind zone caused by the camera's limited field of view at close range, improving the success rate of near-field ingestion.

4.3 Proximity-Based Chassis Navigation Module

This module combines real-time proximity sensing with a closed-loop PID chassis controller, responsible for executing the final stage of ball retrieval: alignment, approach, and physical ingestion. It connects the visual perception pipeline (Section 3.1.6) with the intake actuator (Section 3.X) through a tightly coupled state machine, simultaneously fusing two independent data streams—the image-plane centroid offset from the YOLO detector, and millimeter-resolution distance readings from the RoboMaster EP's onboard infrared sensor.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.3.1 Distance Sensing Submodule

Hardware Interface via RoboMaster-SDK-Ultra.

Distance measurement relies on the RoboMaster EP's built-in infrared proximity sensor, accessed through **RoboMaster-SDK-Ultra**. This SDK is a community-maintained high-performance fork of the official DJI Python SDK, extending compatibility to Python 3.7 through 3.13 and bundling `libmedia_codec_ultra` for zero-copy H.264 decoding. Installation on the Jetson Orin NX proceeds as follows:

```

1  # Automated installation (recommended)
2  git clone https://github.com/RamessesN/Robomaster-SDK-Ultra
3  cd Robomaster-SDK-Ultra
4  sudo chmod 755 ./installer.sh && ./installer.sh
5
6  # Or manual installation on Ubuntu/Debian
7  sudo apt update && sudo apt install ffmpeg libopus-dev
8  pip install -e .
9  cd robomaster_lib/libmedia_codec_ultra && pip install -e .
10 cd ../../pybind11 && pip install -e .

```

After installation, the sensor API remains consistent with the official DJI documentation, but the import must come from the patched namespace:

```

1  from robomaster_ultra import robot  # replaces: from robomaster import robot

```

Subscription Design.

Distance data is obtained through the SDK's asynchronous subscription callback mechanism. A dedicated background thread invokes `sub_distance(freq=10, callback=...)` to register a 10 Hz polling loop; upon each callback trigger, the raw sensor reading (unit: millimetres) is written to a module-level shared variable:

```

1  # src/distance/distance_sub.py
2
3  latest_distance: float | None = None
4
5  def get_distance(ep_sensor):
6      """Register the 10 Hz distance subscription."""
7      ep_sensor.sub_distance(freq=10, callback=sub_data_handler_distance)
8
9  def sub_data_handler_distance(sub_info):
10     """Callback: atomically update the shared distance register."""

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

11     global latest_distance
12     latest_distance = sub_info[0]    # unit: millimetres

```

The sentinel value `None` is used to distinguish “sensor not yet initialized” from a legitimate zero reading. In all chassis controller logic, a `None` reading is substituted with the symbolic constant 8848 (the height of Mount Everest in metres, employed here as an equivalent infinitely-far distance value), ensuring the controller defaults to a pure vision-guided mode in the absence of sensor data.

4.3.2 PID Controller Configuration

Two independent PID controllers respectively govern the two degrees of freedom required during the approach process:

```

1  from simple_pid import PID
2
3  # Lateral alignment: drives image-plane centroid error to zero
4  pid_turn = PID(0.15, 0.03, 0.03, setpoint=0)
5  pid_turn.output_limits = (-50, 50)    # RPM-equivalent units
6
7  # Longitudinal approach: drives sensor distance to 10 mm (contact)
8  pid_forward = PID(0.5, 0.1, 0.05, setpoint=10)
9  pid_forward.output_limits = (-40, 40)

```

- `pid_turn` receives the pixel-space lateral offset `visual_error_x` produced by the YOLO centroid extractor (a positive value indicates the target is to the right of the frame center). Its output is negated before application, such that a target on the right produces a clockwise chassis rotation.
- `pid_forward` receives the infrared distance reading (millimetres) and drives the robot toward a target distance of 10 mm —i.e., physical contact with the ball. When the distance is below 800 mm, this controller overrides the open-loop cruise speed (30 RPM); above 800 mm, the chassis advances at a fixed cruise speed to reduce approach time.

4.3.3 Chasing Sequence State Tracking

The Boolean flag `is_chasing_sequence` decouples the visual locking phase from the physical approach phase. Once the YOLO pipeline sets `target_locked` to `True`, this flag is latched high, and the last valid distance reading is saved to `last_valid_dist`. The flag is cleared only when the vision module issues `scan_needed = True` —indicating complete target loss following a successful ingestion or timeout.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

This design prevents the distance sensor from falsely triggering the intake motors due to irrelevant obstacles briefly entering the sensor field of view before a visual lock has been established.

```

1  if target_valid:
2      is_chasing_sequence = True
3      last_valid_dist = dist
4  elif need_scan:
5      is_chasing_sequence = False
6      last_valid_dist = 8848

```

4.3.4 Multi-Stage Approach & Ingestion Logic

When `is_chasing_sequence` is active, the controller executes three priority-ordered behaviors based on the current sensor reading.

Phase 1 — Cruise Tracking (`dist > 800 mm`).

Both PID loops operate simultaneously. The turn controller corrects lateral deviation; when the centroid error exceeds 40 pixels, the forward speed is limited to 5 RPM to ensure rotational alignment stabilizes before resuming advance:

```

1  turn_speed = -pid_turn(error_x)
2  if abs(error_x) > 40:
3      forward_speed = 5          # prioritise alignment over advance
4  else:
5      if dist < 800:
6          forward_speed = -pid_forward(dist)  # PID-controlled approach
7      else:
8          forward_speed = 30      # open-loop cruise
9  drive_chassis(ep_chassis, forward_speed, turn_speed)

```

Phase 2 — Blind-Zone Relay (`target lost and dist < 450 mm`).

The camera’s minimum focusing distance together with the physical offset between the camera and the intake rollers collectively form a “blind zone” — the ball disappears from the video frame before entering the rollers. The relay logic bridges this blind zone: if the target was last confirmed within 300 mm (`last_valid_dist < 300`) and the distance sensor still reads below 450 mm, the chassis maintains straight forward advance at 30 RPM without visual feedback:

```

1  elif not target_valid and dist < 450 and last_valid_dist < 300:
2      logger.info(

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

3         f"Blind-zone relay "
4         f"(current: {dist/10:.1f} cm, "
5         f"last valid: {last_valid_dist/10:.1f} cm) "
6         "-- maintaining straight advance..."
7     )
8     drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)

```

Phase 3 — Ingestion Trigger ($\text{dist} \leq 200 \text{ mm}$).

When the sensor confirms the ball has entered the 200 mm range, the intake motors pre-spin at full speed and the chassis advances at a fixed 30 RPM. A sliding-window detector then continuously monitors the ball disappearance signal: if the distance reading exceeds 250 mm for 10 consecutive 20 ms cycles ($\text{missing_count} \geq 10$), the ball is determined to have been successfully ingested. A 3-second hard timeout guards against jamming or missing:

```

1  if dist <= 200:
2      logger.info(
3          f"Target entered pre-ingestion zone "
4          f"({dist/10:.1f} cm) -- pre-spinning motors..."
5      )
6      if intake_motor:
7          intake_motor.set_speed(90, 90)
8      drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)
9
10     swallow_start_time = time.time()
11     missing_count = 0
12     is_swallowed = False
13
14     while time.time() - swallow_start_time < 3.0:
15         current_dist = ds.latest_distance \
16             if ds.latest_distance is not None else 8848
17         if current_dist > 250:
18             missing_count += 1
19         else:
20             missing_count = 0
21         if missing_count >= 10:
22             is_swallowed = True
23             break
24         time.sleep(0.02)
25
26     if is_swallowed:
27         logger.info("Ball successfully ingested into storage bay.")
28     else:
29         logger.info("Warning: advance timeout -- possible jam or miss.")

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

30
31     chassis_stop(ep_chassis)
32     if intake_motor:
33         intake_motor.set_speed(0, 0)
34         is_chasing_sequence = False
35         last_valid_dist = 8848
36         time.sleep(0.5)
37         continue

```

4.3.5 Chassis Drive Primitives

The Mecanum wheel drive equation converts the two scalar commands `forward_speed` and `turn_speed` into per-wheel RPM targets. Since the DJI SDK's `drive_wheels(w1, w2, w3, w4)` interface uses the right-front / left-front / left-rear / right-rear wheel ordering convention, the differential mixing is computed as follows:

```

1  def drive_chassis(ep_chassis, forward_speed, turn_speed):
2      left_speed = forward_speed + turn_speed
3      right_speed = forward_speed - turn_speed
4      ep_chassis.drive_wheels(
5          w1=right_speed,    # right-front
6          w2=left_speed,     # left-front
7          w3=left_speed,     # left-rear
8          w4=right_speed     # right-rear
9      )
10
11  def chassis_stop(ep_chassis):
12      ep_chassis.drive_wheels(w1=0, w2=0, w3=0, w4=0)
13      time.sleep(0.02)      # brief settle pause before next command

```

During pure straight-line motion, the two sides receive equal speeds; when `turn_speed` is non-zero, one side's speed increases while the other decreases, producing an in-place rotation superimposed on the translational velocity. The 20 ms settling delay in `chassis_stop` prevents new burst commands from being issued before the previous wheel-speed packet is confirmed by the chassis firmware.

4.3.6 Idle & Search Behavior

When no tracking task is in progress, the controller falls back to two passive behaviors driven by the vision module:

```

1  else:

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

2   if need_scan:
3       # Rotate in place to sweep the camera FOV
4       drive_chassis(ep_chassis, forward_speed=0, turn_speed=30)
5   else:
6       chassis_stop(ep_chassis)

```

The `need_scan` flag is triggered by the finite state machine's `SEARCHING` state, indicating that coverage path following has been temporarily suspended, pending local re-detection. The in-place rotation at a fixed 30 RPM sweeps the 120° camera field of view across the surrounding area; once a new target is detected, the chasing sequence automatically re-engages.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

5 USER INTERFACE DESIGN

This section describes the user interface design of the CourtSweeper iOS application. Developed natively in Swift using the SwiftUI framework, the mobile application serves as the primary remote command center. It connects directly to the Jetson Orin NX's localized Wi-Fi hotspot, providing real-time situational awareness, manual teleoperation capabilities, and system telemetry monitoring.

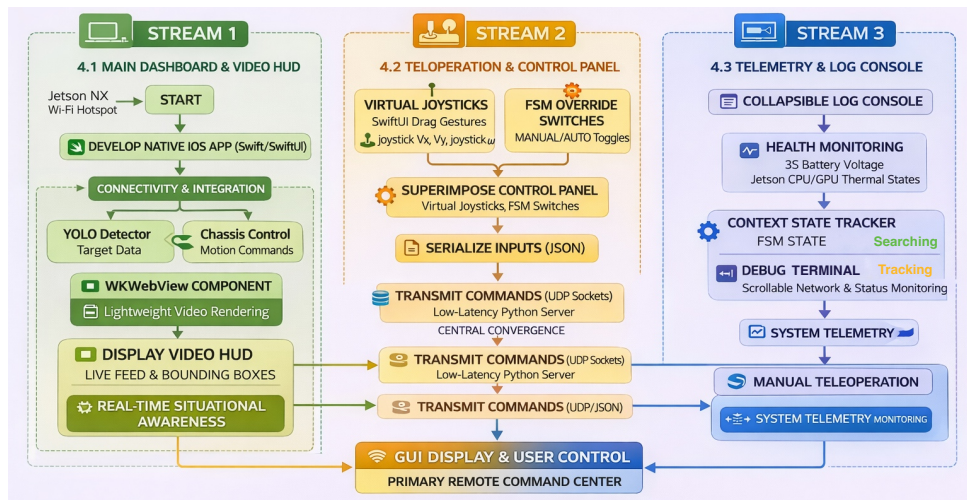


图 12: The software architecture and data flow for the CourtSweeper iOS application, illustrating the centralized SwiftUI core connected via bidirectional Wi-Fi and UDP sockets to external Jetson computing units, and featuring functional breakdowns of main dashboard, teleoperation, and telemetry modules.

5.1 Main Dashboard & Video HUD

The primary interface is designed as a landscape-oriented Heads-Up Display (HUD) to maximize the operator's field of view. The core component of this view is the video presentation layer.

- **Video Streaming Rendering:** Instead of relying on complex and latency-prone H.264 decoders on the client side, the app utilizes a lightweight WKWebView component to render the MJPEG video stream broadcasted by the Jetson unit.
- **AR Overlay:** The AI-processed bounding boxes, target crosshairs, and ball coordinates are overlaid directly onto the video feed, providing the operator with intuitive, real-time feedback on the YOLO perception module's performance.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

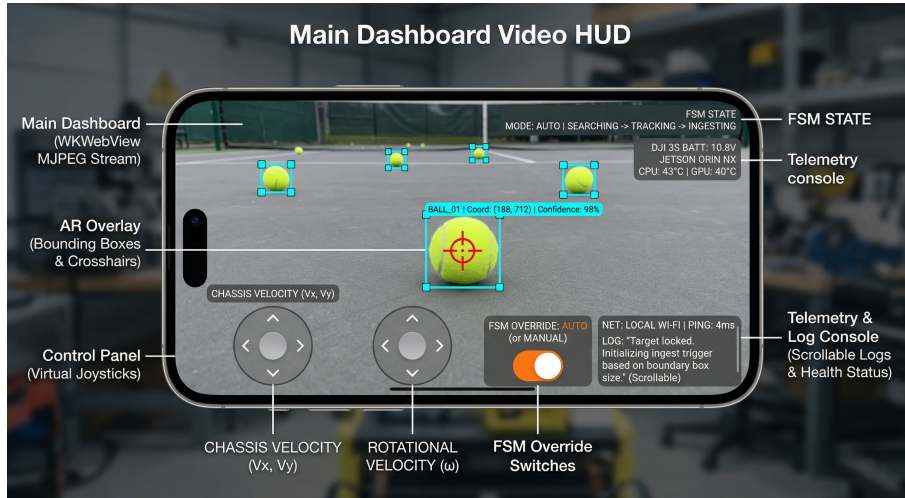


图 13: User interface layout of the CourtSweeper iOS application. The primary HUD renders the low-latency MJPEG video stream overlaid with real-time YOLO bounding boxes. The lower panel provides virtual joysticks for holonomic teleoperation and manual override switches for the system's Finite State Machine (FSM).

5.2 Teleoperation & Control Panel

Superimposed on the lower sections of the HUD is the Control Panel, designed for ergonomic thumb access on an iPhone or iPad.

- **Virtual Joysticks:** Implemented via SwiftUI drag gestures, the left joystick dictates the omnidirectional translational movement (V_x, V_y) of the Mecanum chassis, while the right joystick controls rotational velocity (ω).
- **FSM Override Switches:** A prominent set of toggle buttons allows the user to switch the robot's FSM between MANUAL (teleoperation) and AUTO (autonomous searching and ingesting) states.
- **Command Transmission:** Control inputs are serialized into lightweight JSON packets and transmitted to the Jetson's Python server via low-latency UDP sockets using Apple's native Network framework.

5.3 Telemetry & Log Console

To facilitate debugging and monitor system health during field tests, a collapsible Log Console is integrated into the interface.

- **Health Monitoring:** Displays critical hardware telemetry parsed from the DJI SDK, including the 3S battery voltage (preventing over-discharge) and the Jetson's CPU/GPU thermal states.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

- **State Machine Tracker:** A dynamic label continuously reflects the robot's current FSM state (e.g., *SEARCHING* → *TRACKING* → *INGESTING*).
- **Debug Terminal:** A scrolling text view (implemented via `ScrollView` in SwiftUI) outputs real-time network connection statuses, ping latency, and critical error logs thrown by the Python backend.